arXiv:2311.10702v2 [cs.CL] 20 Nov 2023

Camels in a Changing Climate: Enhancing LM Adaptation with TÜLU 2

Hamish Ivison*♠ Yizhong Wang*♣♠ Valentina Pyatkin♣♠ Nathan Lambert♣
Matthew Peters♣ Pradeep Dasigi♣ Joel Jang ♣♠ David Wadden♣
Noah A. Smith♣♠ Iz Beltagy♣ Hannaneh Hajishirzi♣♠

♣Allen Institute for AI ♠University of Washington
{yizhongw,hamishiv}@cs.washington.edu

Abstract

Since the release of TÜLU [Wang et al., 2023b], open resources for instruction tuning have developed quickly, from better base models to new finetuning techniques. We test and incorporate a number of these advances into TÜLU, resulting in TÜLU 2, a suite of improved TÜLU models for advancing the understanding and best practices of adapting pretrained language models to downstream tasks and user preferences. Concretely, we release: (1) **TÜLU-V2-mix**, an improved collection of high-quality instruction datasets; (2) **TÜLU 2**, LLAMA-2 models finetuned on the V2 mixture; (3) **TÜLU 2+DPO**, TÜLU 2 models trained with direct preference optimization (DPO), including the largest DPO-trained model to date (**TÜLU 2+DPO 70B**); (4) **CODE TÜLU 2**, CODE LLAMA models finetuned on our V2 mix that outperform CODE LLAMA and its instruction-tuned variant, CODE LLAMA-Instruct. Our evaluation from multiple perspectives shows that the TÜLU 2 suite achieves state-of-the-art performance among open models and matches or exceeds the performance of GPT-3.5-turbo-0301 on several benchmarks. We release all the checkpoints, data, training and evaluation code to facilitate future open efforts on adapting large language models.

1 Introduction

The capabilities of large language models (LMs) to follow user requests have been progressing rapidly through a wide range of openly available models, datasets, and training methods. Since the release of the original TÜLU models [Wang et al., 2023b], there have been a number of significant advances in almost all aspects of language model adaptation, from the release of improved finetuning datasets [Ding et al., 2023, Cui et al., 2023], to increasingly powerful base models [Touvron et al., 2023a, Jiang et al., 2023], to powerful and accessible adaptation methods for combining these components [Rafailov et al., 2023, Dettmers et al., 2023]. We comprehensively evaluate and combine these recent advances to present strong open models across 7, 13, and 70 billion parameter scales with empirical studies of various training recipes.

Accompanying our new models, we release a new dataset mixture, **TÜLU-V2-mix** that results in stronger performance across a variety of reasoning and knowledge-probing tasks. We also compare the performance of both new parameter efficient tuning and reinforcement learning from human feedback (RLHF) methods. Included in our model suite is a LLAMA-2 70B model finetuned on TÜLU-V2-mix and further trained using direct preference optimization (DPO) algorithm, representing **the first stable demonstration of using DPO at scales of 70 billion parameters**. This model achieves results competitive with state-of-the-art on the MT-Bench and AlpacaEval benchmarks.

We additionally explore training with quantized low-rank adaptation (QLoRA), finding that it solid performance across traditional language processing tasks, but falls behind on evaluations that ex-

*Equal contribution.

amine long-form text generation such as AlpacaEval. Finally, we apply our mixture to CODE LLAMA [Roziere et al., 2023], resulting in CODE TÜLU 2, which outperforms both the base CODE LLAMAmodel and its instruction-tuned variant CODE LLAMA-Instruct across all model sizes.

TÜLU-2 validates and extends the progress seen across many open instruction model recipes released recently, such as those with some RL component, including Zephyr-Beta [Tunstall et al., 2023], LLAMA-2-chat [Touvron et al., 2023a], XWin [Xwin-LM Team, 2023], WizardLM [Xu et al., 2023], and OpenChat [Wang et al., 2023a], and some without, including MISTRAL-Instruct [Jiang et al., 2023] and Mosaic Pretrained Transformer (MPT) [MosaicML, 2023].

In summary, with TÜLU 2, we find that:

1. Recent **distilled data mixtures have significantly improved** in terms of downstream performance over both instruction and preference datasets available only six months ago, with our new mixture outperforming our old mixture by an average of 8%.
2. **DPO training scales to 70 billion parameter models**, and significantly improves open-ended generation metrics without degrading model capabilities, improving AlpacaEval performance by an average of 13% across model sizes. Our largest DPO trained model, TÜLU 2+DPO 70B, achieves state-of-the-art performance for MT-Bench [Zheng et al., 2023] compared to open-weight models.
3. **QLoRA training does not match full-finetuning in long-form generation tasks**, although the gap shrinks with model size (from 10% worse on average to 3% worse on average across our tasks). We note that QLoRA especially underperforms on open-ended generation tasks such as AlpacaEval (20% average gap in performance).
4. **CODE TÜLU 2 significantly improves coding abilities over TÜLU 2** (70% average improvement in Codex-Eval) but degrades open-ended model generations in AlpacaEval (20% average drop in performance).

We publicly release all models, data, and code associated with this work. Models and the new dataset mix can be found at `https://huggingface.co/collections/allenai/tulu-v2-suite-6551b56e743e6349aab45101`. Our finetuning and evaluation code can be found at `https://github.com/allenai/open-instruct`. We hope that publicly releasing all artifacts aids future research into post-pretraining LM adaptation.

2 TÜLU V2 Details

We first detail the aspects of adaptation we explored for TÜLU 2 in comparison to TÜLU 1 [Wang et al., 2023b]: new base models, a new data mixture, extended context training data, and RLHF training. TÜLU 1 constructed two data instruction mixes through a variety of experiments, one containing prompt-response pairs fully written by humans from the FLAN, Dolly and Open Assistant datasets, and another containing prompt-response pairs fully or partially generated by OpenAI models along with the human-written data.

Improved base models We first switch from using LLAMA-1 models [Touvron et al., 2023a] to LLAMA-2 [Touvron et al., 2023b], a newer set of models following similar architecture to LLAMA-1 but pretrained on significantly more tokens (2 trillion tokens as opposed to 1 or 1.4 trillion tokens), and displaying improved performance (Touvron et al. [2023b] shows a 10% average improvement across model sizes on a set of academic benchmarks). We also experiment with CODE LLAMA, a set of LLAMA-2 models further pretrained on code data. We finetune models at all possible LLAMA-2 sizes: 7B, 13B, and 70B, and all possible CODE LLAMA sizes: 7B, 13B, and 34B.

V2 data mixture Our original data mixture (TÜLU-V1-mix) was based on ablations over human and GPT-generated datasets – we refer readers to Wang et al. [2023b] for a full list. We keep a number of high-quality datasets from our first mix, and add new datasets that are either carefully manually curated for quality or generated from GPT models while encouraging complexity and diversity. We additionally downsample larger datasets such as FLAN to reduce the overall size of the training mixture, and remove Dolly [Databricks, 2023] from the mixture due to its poor performance in previous ablations. Our V2 mixture, TÜLU-V2-mix, comprises of data from the following sources (we mark datasets newly added to our V2 mixture with *):

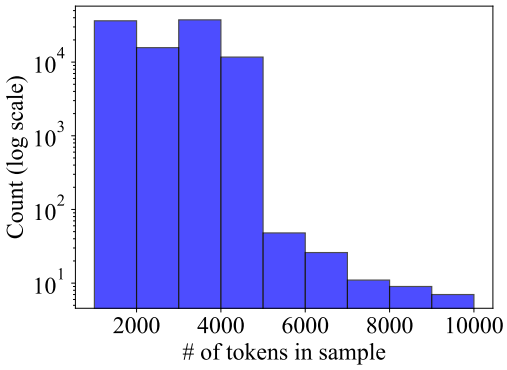

Figure 1: Histogram of token lengths in our V2 data mixture.

- **FLAN** [Chung et al., 2022]: We use 50,000 examples sampled from FLAN v2.
- **CoT**: To emphasize chain-of-thought (CoT) reasoning, we sample another 50,000 examples from the CoT subset of the FLAN v2 mixture.
- **Open Assistant 1** [Köpf et al., 2023]: We isolate the highest-scoring paths in each conversation tree and use these samples, resulting in 7,708 examples. Scores are taken from the quality labels provided by the original annotators of Open Assistant 1.
- **ShareGPT**[2]: We use all 114,046 examples from our processed ShareGPT dataset, as we found including the ShareGPT dataset resulted in strong performance in prior work.
- **GPT4-Alpaca** [Peng et al., 2023]: We sample 20,000 samples from GPT-4 Alpaca to further include distilled GPT-4 data.
- **Code-Alpaca** [Chaudhary, 2023]: We use all 20,022 examples from Code Alpaca, following our prior V1 mixture, in order to improve model coding abilities.
- ***LIMA** [Zhou et al., 2023]: We use 1,030 examples from LIMA as a source of carefully curated data.
- ***WizardLM Evol-Instruct V2** [Xu et al., 2023]: We sample 30,000 examples from WizardLM, which contains distilled data of increasing diversity and complexity.
- ***Open-Orca** [Lian et al., 2023]: We sample 30,000 examples generated by GPT-4 from OpenOrca, a reproduction of Orca [Mukherjee et al., 2023], which augments FLAN data with additional model-generated explanations.
- ***Science literature**: We include 7,544 examples from a mixture of scientific document understanding tasks— including question answering, fact-checking, summarization, and information extraction. A breakdown of tasks is given in Appendix C.
- ***Hardcoded**: We include a collection of 140 samples using prompts such as 'Tell me about yourself' manually written by the authors, such that the model generates correct outputs given inquiries about its name or developers.

Additionally, we filter any samples that include references to other LLM systems such as GPT-4, Open Assistant, or Claude, to avoid contradicting the hardcoded prompts. After filtering, the V2 mixture consists of 326,154 samples, compared to 490,445 in the V1 mixture. Our dataset is available at `https://huggingface.co/datasets/allenai/tulu-v2-sft-mixture`.

Extended context length We expand the context length during training from a maximum of 2,048 tokens to 8,192 tokens in order to make better use of the many lengthy samples in datasets such as

[2] ShareGPT (`https://sharegpt.com/`) data was used to build the Vicuna model [Chiang et al., 2023], but the exact dataset has not been released. Following Wang et al. [2023b], we instead use a reproduced version from `https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered/tree/main/HTML_cleaned_raw_dataset`, and follow Vicuna to split the long conversations into blocks with a maximum length of 4,196 tokens.

ShareGPT and Open Assistant 1. Moving from 2,048 to 8,192 max length means we only truncate 20 (as opposed to 63,900) samples within our V2 mixture, better capturing the long tail of lengthy examples in our training data. We plot the length distribution of our V2 mixture in Figure 1. The mean length of a sample is 1097 tokens, with the 25th and 75th percentile values being 230 and 1464 respectively.

RLHF training Reinforcement learning from human feedback (RLHF) is a core component of modern user-facing LLM systems [Bai et al., 2022, Ouyang et al., 2022, Touvron et al., 2023a]. Early systems for RLHF were built primarily upon the proximal policy optimization (PPO) algorithm, but recent advances have seen exploration of offline RL [Snell et al., 2022], reward model data filtering called rejection sampling (RS) [Touvron et al., 2023a] or reinforced self-training (ReST) [Gulcehre et al., 2023] and direct integration of preference data [Rafailov et al., 2023]. In this work, we use the direct preference optimization (DPO) algorithm due to the simplicity of its implementation [Rafailov et al., 2023]. For DPO training, we follow the Zephyr-Beta approach [Tunstall et al., 2023]: we train on a filtered and binarized form of UltraFeedback [Cui et al., 2023] for three epochs. One thing to note is the low learning rate, 5×10^{-7}, required for stable and effective DPO training. We find this significantly improves performance on open-ended generation evaluations such as AlpacaEval [Li et al., 2023], while making little to no difference in performance over more capability-focussed evaluations such as MMLU and HumanEval.

QLoRA training We experimented with QLoRA training at the instruction tuning stage in order to determine if we could reduce our compute demands without reducing performance. Due to sub-par performance at the instruction tuning stage, we did not explore using QLoRA during RLHF training, although we note that prior work has found it to perform well for PPO-based RLHF training [Santacroce et al., 2023, Sun et al., 2023].

3 Experiments

Evaluation tools We reuse the evaluation framework from TÜLU 1 [Wang et al., 2023b], which includes evaluations testing factual knowledge (MMLU), reasoning (GSM8k, Big Bench Hard), multilinguality (TydiQA), coding (CodexEval), open-ended generation (AlpacaEval), toxicity (ToxiGen), and truthfulness (TruthfulQA). We refer the reader to Wang et al. [2023b] for a more in-depth explanation of these evaluations, and provide an overview of each evaluation in Appendix A.

We make two changes to this evaluation framework: first, we replace our old AlpacaFarm setup with the default AlpacaEval setup [Li et al., 2023], making our reported numbers directly comparable with the AlpacaEval leaderboard (`https://tatsu-lab.github.io/alpaca_eval/`). At time of writing, AlpacaEval does not use a pinned GPT-4 version for evaluation, so we ensure all evaluations reported use GPT-4-0613 as the evaluator model. Second, we also evaluate a set of models on MT-Bench [Zheng et al., 2023], a popular benchmark for open-ended generation that similarly uses GPT-4 to judge model outputs across a diverse set of prompts.

While TruthfulQA is included in our evaluation suite, we found that the data used for DPO training (UltraFeedback) made use of TruthfulQA prompts. As such, we omit TruthfulQA results when showing comparisons with contaminated models (any models trained with the UltraFeedback dataset). We also note that although we report results for several GPT models (GPT-4-0314, GPT-3.5-turbo-0301, GPT-4-1106-preview), **we cannot rule out the possibility they are trained on the evaluation benchmark datasets**.

Training We detail the hyperparameters used to train models in Appendix B. The 70B variant of TÜLU V2-DPO was trained on a 512-core TPUv3, completing three epochs in approximately 7 days.

3.1 Overall Results

We present our overall results comparing TÜLU-2 to popular proprietary and open models in Table 1. We find that:

TÜLU 2 outperforms all open models on average. TÜLU-2 70B is the highest-performing model on average and is the best-performing open model in 3/7 tasks. For the remaining 4 tasks, it is

	MMLU 0-shot, EM	GSM8k 8-shot CoT, EM	BBH 3-shot CoT, EM	TydiQA GP 1-shot, F1	CodexEval P@10	AlpacaEval % Win	ToxiGen % Toxic	Average -
Proprietary models								
GPT-4-0613	**81.4**	**95.0**	**89.1**	**65.2**	87.0	91.2	0.6	**86.9**
GPT-3.5-turbo-0613	65.7	76.5	70.8	51.2	88.0	**91.8**	**0.5**	77.6
GPT-3.5-turbo-0301	67.9	76.0	66.1	51.9	**88.4**	83.6	27.7	72.3
Non-TÜLU Open Models								
Zephyr-Beta 7B	58.6	28.0	44.9	23.7	54.3	86.3	64.0	47.4
Xwin-LM v0.1 70B	**65.0**	**65.5**	**65.6**	38.2	**66.1**	<u>**95.8**</u>	12.7	**69.1**
LLAMA-2-Chat 7B	46.8	12.0	25.6	22.7	24.0	87.3	<u>**0.0**</u>	45.4
LLAMA-2-Chat 13B	53.2	9.0	40.3	32.1	33.1	91.4	<u>**0.0**</u>	51.3
LLAMA-2-Chat 70B	60.9	59.0	49.0	**44.4**	52.1	94.5	<u>**0.0**</u>	65.7
TÜLU 2 Suite								
TÜLU 2 7B	50.4	34.0	48.5	46.4	36.9	73.9	7.0	54.7
TÜLU 2+DPO 7B	50.7	34.5	45.5	44.5	40.0	85.1	0.5	56.3
TÜLU 2 13B	55.4	46.0	49.5	53.2	49.0	78.9	1.7	61.5
TÜLU 2+DPO 13B	55.3	49.5	49.4	39.7	48.9	89.5	1.1	61.6
TÜLU 2 70B	67.3	<u>**73.0**</u>	<u>**68.4**</u>	<u>**53.6**</u>	68.5	86.6	0.5	<u>**73.8**</u>
TÜLU 2+DPO 70B	<u>**67.8**</u>	71.5	66.0	35.8	<u>**68.9**</u>	**95.1**	**0.2**	72.1

Table 1: The evaluation metrics of our core TÜLU-2 suite and its peers. Most of the models included use LLAMA 2 base models, except Zephyr-Beta, which uses MISTRAL-7B. For all evaluations except ToxiGen, higher scores are better. We average scores naively, apart from Toxigen, where we take 100 - x as the value to average. The top-performing open model per task has been underlined, and the top-performing model in each set of models is bolded.

Size	Data	MMLU 0-shot	GSM8k 8-shot CoT	BBH 3-shot CoT	TydiQA 1-shot	Codex-Eval Pass@10	AlpacaEval %win	ToxiGen % Toxic	TruthfulQA %Info+True	Average -
7B	ShareGPT	47.8	20.0	41.5	24.0	29.2	72.3	12.6	**54.1**	47.0
	V1 mix.	49.2	**37.0**	44.2	**52.9**	33.9	64.5	39.9	40.8	47.8
	V2 mix.	**50.4**	34.0	**48.5**	46.4	**36.9**	**73.9**	**7.0**	50.2	**54.2**
13B	V1 mix.	52.3	**53.0**	**50.6**	**58.8**	38.9	67.7	18.7	45.3	56.0
	V2 mix.	**55.4**	46.0	49.5	53.2	**49.0**	**78.9**	**1.7**	**55.8**	**60.8**
70B	V1 mix.	**67.3**	**74.5**	67.5	**56.8**	65.4	82.8	**0.0**	57.9	71.5
	V2 mix.	**67.3**	73.0	**68.4**	53.6	**68.5**	**86.6**	0.5	**62.2**	**72.4**

Table 2: Results of LLAMA-2 models finetuned on our V1 and V2 data mixtures, and ShareGPT.

outperformed in MMLU and CodexEval by TÜLU 2+DPO 70B, in ToxiGen by LLAMA-2-Chat models, and in AlpacaEval by Xwin-LM 70B. We note that the average gap between TÜLU 2 70B and the highest performing model in these 4 tasks is under 1%, highlighting that TÜLU 2 is at least competitive if not outright better than all open models in most evaluations.

TÜLU 2 is competitive with GPT 3.5-0301. TÜLU 2 70B achieves similar performance to GPT-3.5-turbo-0301 in MMLU, BBH and TydiQA, and outperforms it in AlpacaEval and ToxiGen. However, there remains a large gap with GPT-4 and a moderate gap with GPT-3.5-turbo-0613 (a more modern variant of the model) in most evaluations.

Scaling trends remain strong with TÜLU 2. Increasing model size improves almost every metric when the finetuning setup is held consistent across our model suite.

3.2 TÜLU V1 vs V2 Data Mixtures

We compare our new model suite to our old models in Table 2, comparing LLAMA-2 models at all sizes on our V1 and V2 mix. We additionally compare our V2 mix to a model trained only on ShareGPT, the most promising single dataset from our original work. We find that:

Models trained on the V2 mix perform better than models trained on the V1 mix on open-ended generation. V2 mix models outperform V1 mix models consistently on BBH, Codex-Eval,

AlpacaEval, and TruthfulQA, and consistently underperform the V1 mix on GSM8k and TydiQA. The former is likely due to training on fewer CoT examples (which contains the GSM8k train dataset), while the latter indicates our V2 mix is worse for multilingual capabilities. This reinforces the findings from Wang et al. [2023b] that no one dataset is optimal for all tasks, although we note on average models trained on our V2 mix outperform those trained on our V1 mix.

Models trained on the V2 mix outperform training on ShareGPT across most evals. In prior work and in Table 2, we find that training on ShareGPT alone results in overall performance close to models trained on our V1 mix, and greatly improved AlpacaEval performance. However, our new mix actually outperforms using ShareGPT alone both overall and only considering AlpacaEval. This is likely due to the V2 mix's greater reliance on distilled datasets that have similar origins to ShareGPT.

Improvements from the V2 mix shrink with model size. While the V2 mix provides a 13% average improvement at the 7B scale, it only provides a 1% improvement at the 70B scale. This suggests that the importance of instruction data quality may shrink as model size (and/or capabilities) increase.

Having established the overall superiority of our V2 mix, especially on open-ended generation, we now turn to alternate finetuning methods to further improve TÜLU 2.

3.3 Scaling DPO Training

Size	Model	MMLU 0-shot	GSM8k 8-shot CoT	BBH 3-shot CoT	TydiQA 1-shot	Codex-Eval Pass@10	AlpacaEval %win	ToxiGen % Toxic	Average
7B	TÜLU 2	50.4	34.0	**48.5**	**46.4**	36.9	73.9	7.0	54.7
	TÜLU 2+DPO	**50.7**	**34.5**	45.5	44.5	**40.0**	**85.1**	**0.5**	**56.3**
	Δ	+0.3	+0.5	-3.0	-1.9	+3.1	+11.2	-6.5	+1.6
13B	TÜLU 2	**55.4**	46.0	**49.5**	**53.2**	**49.0**	78.9	1.7	61.5
	TÜLU 2+DPO	55.3	**49.5**	49.4	39.7	48.9	**89.5**	**1.1**	**61.6**
	Δ	-0.1	+3.5	-0.1	-13.5	-0.1	+10.6	-0.6	+0.1
70B	TÜLU 2	67.3	**73.0**	**68.4**	**53.6**	68.5	86.6	0.5	**73.8**
	TÜLU 2+DPO	**67.8**	71.5	66.0	35.8	**68.9**	**95.1**	**0.2**	72.1
	Δ	+0.5	-1.5	-2.4	-17.8	+0.4	+8.5	-0.3	-1.7

Table 3: Evaluation results for TÜLU V2 models with and without DPO finetuning, and the difference between the two results (Δ).

We finetune our models using DPO [Rafailov et al., 2023] and the Ultrafeedback dataset [Cui et al., 2023], following the hyperparameters and overall setup used by Zephyr-Beta [Tunstall et al., 2023], who apply DPO to a 7B Mistral model finetuned on UltraChat [Ding et al., 2023]. Surprisingly, we find these hyperparameters scale, providing stable training and performance improvements for models at all sizes. We show our results in Table 3 and results focusing on GPT-based evaluations (MT-Bench and AlpacaEval) in Table 4. We provide full MT-Bench results in Appendix D. We find that:

DPO training significantly improves AlpacaEval and MT-Bench performance. At all sizes, DPO training provides significant improvements in AlpacaEval, with our largest DPO-trained model significantly outperforming GPT-3.5-turbo-0314 (89.4 vs. 95.1) and is competitive with GPT-4 (see Table 4. **TÜLU 2+DPO 70B is the second best-performing open model on AlpacaEval**,[3] just behind Xwin-LM 70B. We also observe that DPO training provides a large boost in MT-Bench performance for the 13B and 70B size models, with **TÜLU 2+DPO 70B being the best-performing open model compared to all other models on the MT-Bench leaderboard**.[4] Curiously, while TÜLU 2 outperforms most GPT models we examine in AlpacaEval, it underperforms compared to all of them in MT-Bench.

[3] At time of writing. See `https://tatsu-lab.github.io/alpaca_eval/`

[4] At time of writing. See `https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard`

Size	Model	MT-Bench	AlpacaEval	
		Average Score	Winrate (%)	Avg. Output Length
unk.	GPT-4-1106-preview	**9.26**	**97.1**	2041
	GPT-4-0613	9.18	91.2	1090
	GPT-3.5-turbo-0613	8.39	91.8	1416
	GPT-3.5-turbo-0301	7.94	83.6	838
7B	Zephyr-Beta	**7.35**	**86.3**	2721
	TÜLU 2	6.30	73.9	1248
	TÜLU 2+DPO	6.27	85.1	1437
13B	Xwin v0.2	**7.01**	**91.0**	2748
	TÜLU 2	6.70	78.9	1034
	TÜLU 2+DPO	7.00	89.5	1414
70B	Xwin v0.1	7.53	**95.8**	1797
	TÜLU 2	7.49	86.6	1011
	TÜLU 2+DPO	**7.89**	95.1	1414

Table 4: MT-Bench and AlpacaEval results, along with average output length of AlpacaEval responses. GPT model size is unknown. We include output length to observe the effect of DPO on model verbosity. 'GPT-4-1106-preview' is also known as 'GPT-4 Turbo' (See `https://help.openai.com/en/articles/8555510-gpt-4-turbo`).

DPO training is stable at large scales. We find that DPO training scales without issues with 70B-size models, with DPO training still providing large benefits for open-ended generation (AlpacaEval) even at the 70B size. This suggests DPO is a promising path for training large models on human feedback without the engineering complexity required by PPO. To our knowledge, TÜLU 2+DPO 70B is the largest publicly-released DPO-trained model.

DPO does not dramatically harm most other metrics. We find that DPO training does not significantly change performance in most other metrics we measure, such as factual reasoning (MMLU) or reasoning (BBH, GSM8k), with the exception of multilinguality (which we discuss below). This suggests that DPO training does not significantly change model capabilities.

DPO training significantly drops multilingual capabilities. We find that DPO training significantly drops performance in TydiQA, which tests the multilingual capabilities of our model. However, we note that both our supervised finetuning and DPO data mixes do not explicitly contain multilingual data, and are majority English-language. As such, DPO training is likely to make multilingual outputs further out-of-distribution, and mixing in multilingual data at instruction tuning and DPO training stages may significantly improve these results.

DPO training increases model verbosity. As seen in Table 4, TÜLU 2+DPO models generally output answers of longer length than those trained without DPO. This is in line with prior work showing a bias toward verbosity from RLHF training [Dubois et al., 2023, Singhal et al., 2023]. However, we note that our DPO-trained models appear dramatically less verbose than other open-weight models, which future work will investigate.

3.4 Parameter-efficient Finetuning

In order to reduce compute demands, we experimented with using quantized low-rank adaptation (QLoRA) [Dettmers et al., 2023] at the instruction tuning stage. We followed the suggested hyperparameters from Dettmers et al. [2023] and trained LLAMA-2 models at all sizes using QLoRA. We compare these to our fully-finetuned TÜLU 2 models (without DPO) in Table 5. We find:

QLoRA struggles on open-ended generation tasks. We observe that QLoRA underperforms full-finetuning in AlpacaEval in a consistent manner, likely due to the open-ended nature of the task.

Size	Model	MMLU 0-shot	GSM8k 8-shot CoT	BBH 3-shot CoT	TydiQA 1-shot	Codex-Eval Pass@10	AlpacaEval %win	ToxiGen % Toxic	TruthfulQA %Info+True	Average
7B	LLAMA-2 base	41.8	12.0	39.3	**51.2**	26.8	-	77.3	26.7	-
	TÜLU 2	**50.4**	**34.0**	**48.5**	46.4	**36.9**	**73.9**	**7.0**	40.8	**53.0**
	TÜLU 2 (QLoRA)	48.8	20.5	45.7	49.2	31.7	56.1	14.7	**44.6**	47.7
13B	LLAMA-2 base	52.0	25.0	48.9	**56.5**	32.5	-	85.7	31.1	-
	TÜLU 2	**55.4**	**46.0**	49.5	53.2	**49.0**	**78.9**	1.7	**55.8**	**60.8**
	TÜLU 2 (QLoRA)	54.6	36.0	**52.5**	54.6	39.1	65.6	**0.0**	55.2	57.2
70B	LLAMA-2 base	64.5	55.5	66.0	**62.6**	60.1	-	84.2	38.2	-
	TÜLU 2	67.3	**73.0**	68.4	53.6	**68.5**	**86.6**	**0.5**	**62.2**	**73.4**
	TÜLU 2 (QLoRA)	**67.4**	64.5	**71.6**	60.9	66.9	78.6	**0.5**	58.4	71.0

Table 5: Results from LLAMA-2 models finetuned with and without QLoRA on our V2 mix. We also report results from LLAMA-2 models without any finetuning (base).

We suggest the discrepancy of our results compared to Dettmers et al. [2023] may be due to the wider set of tasks in our evaluation suite, as Dettmers et al. [2023] focusses on MMLU performance as a way to compare QLoRA and full-finetuning performance (where we do see much closer performance between QLoRA and full-finetuning). In our overall average, we observe a gap between QLoRA and full-finetuning.

The gap between QLoRA and full-finetuning shrinks with size. Similar to prior work in parameter-efficient learning [Lester et al., 2021], we find that the average gap in performance between QLoRA and full-finetuning shrinks with model size, suggesting that QLoRA may start to match full-finetuning at even larger model sizes.

3.5 Improving Code Performance with CODE LLAMA

Size	Model	MMLU 0-shot	GSM8k 8-shot CoT	BBH 3-shot CoT	TydiQA 1-shot	Codex-Eval Pass@10	AlpacaEval %win	ToxiGen % Toxic	TruthfulQA %Info+True	Average
7B	CODE LLAMA base	33.8	12.0	43.4	47.6	58.7	-	81.5	26.1	-
	CODE LLAMA Instruct	41.5	17.0	38.4	41.6	64.1	71.9	**1.0**	15.2	48.6
	TÜLU 2	**50.4**	**34.0**	48.5	46.4	36.9	**73.9**	7.0	**40.8**	53.0
	CODE TÜLU 2	43.7	33.0	**49.1**	**52.6**	**68.9**	58.0	5.0	33.0	**54.2**
13B	CODE LLAMA base	37.5	22.0	49.5	52.1	69.8	-	77.9	26.9	-
	CODE LLAMA Instruct	43.3	23.0	48.0	37.8	69.2	75.3	**0.0**	38.1	54.3
	TÜLU 2	**55.4**	**46.0**	49.5	53.2	49.0	**78.9**	1.7	**55.8**	**60.8**
	CODE TÜLU 2	45.9	41.0	**52.8**	**55.7**	**76.2**	64.1	**0.0**	36.7	59.1
34B	CODE LLAMA base	47.4	35.0	57.0	57.1	77.6	-	88.3	24.4	-
	CODE LLAMA Instruct	50.9	38.0	59.2	55.1	76.5	**84.5**	**0.0**	**51.2**	64.4
	CODE TÜLU 2	**53.6**	**54.0**	**64.3**	**60.6**	**82.5**	76.8	**0.0**	42.0	**66.7**

Table 6: Evaluation results comparing models based on CODE LLAMA with our TÜLU models. CODE TÜLU 2 refers to CODE LLAMA models finetuned on our V2 mixture.

Finally, we attempted using CODE LLAMA [Roziere et al., 2023] as a base model instead of LLAMA-2 due to its improved performance on coding tasks. We dub CODE LLAMA models trained on our V2 data mixture as CODE TÜLU 2 models. We present our results comparing CODE LLAMA and LLAMA-2 models fully finetuned on our V2 mixture in Table 6. We find that:

CODE TÜLU 2 models significantly outperform TÜLU 2 models at coding tasks. As expected, CODE TÜLU 2 models report drastically improved Codex-Eval performance compared to TÜLU 2 – in Codex-Eval, our smallest (7B) CODE TÜLU 2 model matches the performance of TÜLU-V2+DPO 70B, our strongest LLAMA-2-based model. This highlights the efficacy of using smaller, domain-specific models when limiting evaluation to that domain alone.

CODE TÜLU 2 and TÜLU 2 display drastically different results across non-code evaluations. While we can only compare two sizes, we find that TÜLU 2 models consistently outperform CODE TÜLU 2 models in 4 out of 8 tasks (MMLU, GSM8k, AlpacaEval, TruthfulQA), while CODE TÜLU 2 performs well in BBH, TydiQA, ToxiGen, and Codex-Eval. Since CODE LLAMA models are variants

of LLAMA-2 models additionally pretrained on code data, this suggests the continued code pretraining has significantly altered model capabilities. In particular, we note that performance on AlpacaEval appears to drop by a large margin (by around 20%).

Code TÜLU 2 outperforms CODE LLAMA-base and CODE LLAMA-Instruct across all sizes. We find that CODE TÜLU 2 models, using our V2 data mix, outperform both base CODE LLAMA and CODE LLAMA-Instruct models in 5 our of 8 evaluation settings (and are stronger on average), highlighting the efficacy of our V2 data mixture. CODE LLAMA-Instruct was finetuned on an internally developed private dataset we do not have access to, which makes it difficult to compare to our mixture, but the strong performance of CODE LLAMA-Instruct on AlpacaEval suggests the mixture may focus on general open-ended queries rather than specific model capabilities.

We release our CODE TÜLU 2 models alongside the rest of our V2 suite.

4 Conclusion

We present TÜLU 2, a set of models, along with recipes for continuing the progress of fine-tuning LMs across a variety of tasks. This release represents a strong incremental step through better performance of the new data mixture, stability of DPO training, and comparison to parameter-efficient training methods.

Substantial work is still needed to understand the mechanisms causing the improvement in performance from these datasets and the DPO training methodology. Future work could involve more investigation of the impact of methods such as DPO on handling refusal behaviour, investigating the impact of different data ablations on DPO performance, and performing comparisons to other RLHF algorithms (e.g., PPO) at scale. Additionally, incorporating improved base models will likely yield further gains over the models presented here. We hope such work can be enabled by the public release of all our data, code, and models.

Acknowledgments

Research supported by Cloud TPUs from Google's TPU Research Cloud (TRC). We thank Eric Mitchell and Rafael Rafailov for helpful discussions involving DPO training dynamics.

References

R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.

Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

I. Cachola, K. Lo, A. Cohan, and D. Weld. TLDR: Extreme summarization of scientific documents. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4766–4777, Online, Nov. 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.428. URL `https://aclanthology.org/2020.findings-emnlp.428`.

S. Chaudhary. Code alpaca: An instruction-following llama model for code generation. GitHub repository, 2023. URL `https://github.com/sahil280114/codealpaca`.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. Blog post, March 2023. URL `https://lmsys.org/blog/2023-03-30-vicuna/`.

H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

G. Cui, L. Yuan, N. Ding, G. Yao, W. Zhu, Y. Ni, G. Xie, Z. Liu, and M. Sun. Ultrafeedback: Boosting language models with high-quality feedback. *arXiv preprint arXiv:2310.01377*, 2023.

P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4599–4610, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.365. URL https://aclanthology.org/2021.naacl-main.365.

Databricks. Free dolly: Introducing the world's first truly open instruction-tuned llm. Blog post, 2023. URL https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm.

T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

N. Ding, Y. Chen, B. Xu, S. Hu, Y. Qin, Z. Liu, M. Sun, and B. Zhou. Ultrachat: A large-scale auto-generated multi-round dialogue data. GitHub Repository, 2023. URL https://github.com/thunlp/ultrachat.

Y. Dubois, X. Li, R. Taori, T. Zhang, I. Gulrajani, J. Ba, C. Guestrin, P. Liang, and T. B. Hashimoto. Alpacafarm: A simulation framework for methods that learn from human feedback. *arXiv preprint arXiv:2305.14387*, 2023.

X. Geng. Easylm: A simple and scalable training framework for large language models, 2023. URL https://github.com/young-geng/EasyLM.

C. Gulcehre, T. L. Paine, S. Srinivasan, K. Konyushkova, L. Weerts, A. Sharma, A. Siddhant, A. Ahern, M. Wang, C. Gu, et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

T. Hartvigsen, S. Gabriel, H. Palangi, M. Sap, D. Ray, and E. Kamar. TOXIGEN: Controlling Language Models to Generate Implied and Adversarial Toxicity. In *ACL*, 2022. URL https://arxiv.org/abs/2203.09509.

A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

A. Köpf, Y. Kilcher, D. von Rütte, S. Anagnostidis, Z.-R. Tam, K. Stevens, A. Barhoum, N. M. Duc, O. Stanley, R. Nagyfi, et al. Openassistant conversations–democratizing large language model alignment. *arXiv preprint arXiv:2304.07327*, 2023.

E. Lehman, J. DeYoung, R. Barzilay, and B. C. Wallace. Inferring which medical treatments work from reports of clinical trials. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3705–3717, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1371. URL https://aclanthology.org/N19-1371.

B. Lester, R. Al-Rfou, and N. Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic, Nov. 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.243. URL https://aclanthology.org/2021.emnlp-main.243.

X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, and T. B. Hashimoto. Alpacaeval: An automatic evaluator of instruction-following models. Github repository, 2023. URL https://github.com/tatsu-lab/alpaca_eval.

W. Lian, B. Goodson, E. Pentland, A. Cook, C. Vong, and "Teknium". Openorca: An open dataset of gpt augmented flan reasoning traces. https://https://huggingface.co/Open-Orca/OpenOrca, 2023.

S. Lin, J. Hilton, and O. Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, 2022.

Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi. Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3219–3232, Brussels, Belgium, Oct.-Nov. 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1360. URL https://aclanthology.org/D18-1360.

MosaicML. Introducing mpt-7b: A new standard for open-source, commercially usable llms. Blog post, 2023. URL https://www.mosaicml.com/blog/mpt-7b.

S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah. Orca: Progressive learning from complex explanation traces of gpt-4, 2023.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems* (NeurIPS), 2022.

B. Peng, C. Li, P. He, M. Galley, and J. Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.

R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

M. Santacroce, Y. Lu, H. Yu, Y. Li, and Y. Shen. Efficient rlhf: Reducing the memory usage of ppo, 2023.

P. Singhal, T. Goyal, J. Xu, and G. Durrett. A long way to go: Investigating length correlations in rlhf. *arXiv preprint arXiv:2310.03716*, 2023.

C. Snell, I. Kostrikov, Y. Su, M. Yang, and S. Levine. Offline rl for natural language generation with implicit language q learning. *arXiv preprint arXiv:2206.11871*, 2022.

S. Sun, D. Gupta, and M. Iyyer. Exploring the impact of low-rank adaptation on the performance, efficiency, and regularization of rlhf, 2023.

M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.

H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.

H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.

L. Tunstall, E. Beeching, N. Lambert, N. Rajani, K. Rasul, Y. Belkada, S. Huang, L. von Werra, C. Fourrier, N. Habib, et al. Zephyr: Direct distillation of lm alignment. *arXiv preprint arXiv:2310.16944*, 2023.

D. Wadden, S. Lin, K. Lo, L. L. Wang, M. van Zuylen, A. Cohan, and H. Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, Online, Nov. 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.609. URL `https://aclanthology.org/2020.emnlp-main.609`.

G. Wang, S. Cheng, X. Zhan, X. Li, S. Song, and Y. Liu. Openchat: Advancing open-source language models with mixed-quality data. *arXiv preprint arXiv:2309.11235*, 2023a.

Y. Wang, H. Ivison, P. Dasigi, J. Hessel, T. Khot, K. R. Chandu, D. Wadden, K. MacMillan, N. A. Smith, I. Beltagy, et al. How far can camels go? exploring the state of instruction tuning on open resources. *arXiv preprint arXiv:2306.04751*, 2023b.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.

Xwin-LM Team. Xwin-lm, 2023. URL `https://github.com/Xwin-LM/Xwin-LM`.

L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *NeurIPS Datasets and Benchmarks Track*, 2023.

C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, et al. Lima: Less is more for alignment. *arXiv preprint arXiv:2305.11206*, 2023.

A Evaluation Suite

We describe our evaluation suite below for easy reference:

- **MMLU**: We use the official MMLU evaluation script and prompts available at `https://github.com/hendrycks/test`, with modifications to allow for batch processing. We evaluate using 0 few-shot examples, following the original setup of MMLU. We report average accuracy across test examples.
- **GSM**: We evaluate models on the test set of GSM. Following Wei et al. [2022], we evaluate with chain-of-thought. We use 8 few-shot in-context examples. Because all answers in GSM are numbers, we extract the last number in the model response as the final answer. We report average accuracy across test examples.
- **BBH**: We follow the setup described in the original paper Suzgun et al. [2022], and evaluate with chain-of-thought. The officially provided prompts, which have 3 few-shot in-context examples are used. For the CoT setup, we extract the first word after the phrase 'So the answer is', or the entire response if there is no such substring present. We report average accuracy over sub-tasks (all of which use accuracy as the primary metric).
- **TydiQA**: We follow the setup described in the PaLM 2 technical report [Anil et al., 2023] to evaluate models' performance in answering multilingual questions. We report only one setting, GP, where the gold passage that contains the answer is given (GoldP/GP). One in-context example is used to familiarize the model with the answering format.
- **Codex-Eval**: We use the HumanEval dataset in the Codex paper [Chen et al., 2021] for evaluating models' coding ability. The dataset contains 164 programming problems, where models are prompted to complete the Python function given its docstring. Following the original paper, we compute unbiased estimates of pass@k to measure the functional correctness of models' outputs. We report pass@10. We sample with a temperature of 0.8.
- **ToxiGen**: We follow the setup in Touvron et al. [2023b], but use the original set of prompts from Hartvigsen et al. [2022], which are designed to elicit toxic generations for certain groups. We take only the prompts designed to produce toxic language ('hateful' prompts) and use 500 prompts per group to reduce evaluation costs. For base language models, we pass in the original ToxiGen prompts unchanged and greedily decode up to the first new line (or a maximum of 512 tokens). For instruction-tuned models, we place the prompt in the corresponding template, and ask the model to complete the prompt, until the model generates a stop token (or a maximum of 512 tokens). We pass the generated text into a roberta-large model trained to detect toxic content finetuned as part of Hartvigsen et al. [2022][5]. We then report the percentage of generations deemed toxic by the classifier.
- **TruthfulQA**: Following Touvron et al. [2023b], we mainly use the generation setting of TruthfulQA [Lin et al., 2022]. The TruthfulQA dataset contains 818 questions, which are used to prompt the tested model to generate answers. We use the default QA prompt format with 6 in-context QA examples. We follow the official script in their official implemention[6] to do greedy decoding and answer postprocessing. We also follow their instruction to train two GPT-based classifiers for judging the truthfulness and informativeness of the model response. We report the rate of the responses being truthful and informative (% Informative and Truthful) following Touvron et al. [2023b]. We only report the % Informative and Truthful as our primary metric.
- **AlpacaEval**: We use the package provided by Li et al. [2023], following the default setup which asks the evaluated model to generate responses for 805 prompts and employ GPT-4 to compare the response with Davinci-003. We employ the "alpaca_eval_gpt4" annotator. We allow the evaluated model to generate up to 8192 tokens, without specifying special stop sequences. The reported win-rate is the percentage of model generations that GPT-4 reports as being preferred over the generations from Davinci-003.
- **MT-Bench**: We use the single-answer grading setting of MT-Bench, as suggested by the MT-Bench repository[7]. MT-Bench consists of 80 questions with followups, resulting in 160 responses being

[5] https://huggingface.co/tomh/toxigen_roberta
[6] https://github.com/sylinrl/TruthfulQA/
[7] https://github.com/lm-sys/FastChat/tree/main/fastchat/llm_judge#mt-bench

graded by a GPT-4 model across varying domains. While MT-Bench does not have a pinned GPT-4 version, we ensure all reported evaluations use GPT-4-0613.

B Training Hyperparameters

For instruction-tuning/supervised fine-tuning, our training hyperparameters were as follows:

- Precision: BFloat16
- Epochs: 2
- Weight decay: 0
- Warmup ratio: 0.03
- Learning rate: 2e-5 (1e-5 for 70B)
- Max. seq. length: 8,192
- Effective batch size: 128

For QLoRA training, we used the following:

- Epochs: 5
- Weight decay: 0
- Warmup ratio: 0.03
- Learning rate: 1e-4
- Max. seq. length: 4,096
- Effective batch size: 128
- LoRA Rank: 64
- LoRA Alpha: 16
- LoRA dropout: 0.1
- Layers wrapped: all attention and feedforward linear layers

We experimented with a variety of QLoRA hyperparameters and found in smaller-scale experiments that these were the best hyperparameters we could fit into our compute budget while still giving strong performance.

For DPO, we used the following hyperparameters:

- Precision: BFloat16
- Epochs: 3
- Weight decay: 0
- Warmup ratio: 0.1
- Learning rate: 5e-7
- Max. seq. length: 8,192
- Effective batch size: 32
- Beta: 0.1

All models except QLoRA models were trained on a 256-chip (512-chip for 70B DPO training) TPU v3 pod. Our training code is based off EasyLM [Geng, 2023] and available at https://github.com/hamishivi/EasyLM.

QLoRA models were trained on an internal A100 80GB cluster using finetuning code available at https://github.com/allenai/open-instruct.

C Science Mixture Breakdown

We provide a breakdown of what tasks are included, and their dataset of origin, in our science mixture in Table 7.

Dataset	Tasks	# Examples
Evidence Inference [Lehman et al., 2019]	Information extraction: Medical evidence 5-tuples	1,678
Qasper [Dasigi et al., 2021]	Question answering	2,255
SciERC [Luan et al., 2018]	Information extraction: Named entity recognition, Relation extraction	700
SciFact [Wadden et al., 2020]	Fact checking	919
SciTLDR [Cachola et al., 2020]	Summarization	1,992

Table 7: Datasets included in the science literature instruction mix for TÜLU V2.

D Full MT-Bench Results

In Table 8 we show full MT-Bench results, split by category, for all models shown in Table 4. We use GPT-4-0613 as the judge model.

	STEM	Humanities	Reasoning	Coding	Math	Extraction	Roleplay	Writing	Average
Proprietary Models									
GPT-4-1106-preview	**9.90**	**9.95**	8.10	**9.05**	7.95	**9.90**	**9.50**	**9.70**	**9.26**
GPT-4-0613	9.65	9.85	**9.30**	8.60	**8.10**	9.35	9.03	9.55	9.18
GPT-3.5-turbo-0613	9.55	**9.95**	6.20	7.05	7.05	9.00	8.65	9.65	8.39
GPT-3.5-turbo-0301	9.05	9.55	6.30	6.70	5.20	8.60	8.55	9.60	7.94
Open Models									
LLAMA-2-Chat 7B	8.65	8.75	4.25	3.00	2.40	6.50	7.70	8.90	6.27
LLAMA-2-Chat 13B	8.63	9.75	5.10	3.00	3.45	6.93	7.50	8.85	6.65
LLAMA-2-Chat 70B	8.93	9.63	5.80	3.15	3.30	7.25	7.50	9.30	6.86
Zephyr-Beta 7B	9.03	9.63	5.60	**5.10**	**4.45**	7.45	8.20	9.35	7.35
Xwin 70b v0.1	**9.68**	**9.95**	**6.55**	4.25	3.30	**8.75**	8.25	**9.55**	**7.53**
Xwin 13b v0.2	9.55	9.88	5.20	3.60	2.85	7.70	**8.60**	8.68	7.01
TÜLU V2 Models									
TÜLU 2 7B	8.00	9.50	4.40	3.40	3.30	6.10	7.63	8.10	6.30
TÜLU 2+DPO 7B	8.23	9.60	4.30	3.32	2.35	6.05	7.95	8.35	6.27
TÜLU 2 13B	8.70	9.25	5.45	4.30	3.75	7.35	7.50	7.30	6.70
TÜLU 2+DPO 13B	**9.08**	9.80	5.30	3.60	2.95	8.00	8.60	8.70	7.00
TÜLU 2 70B	9.00	9.75	5.50	**5.10**	**4.70**	8.45	8.30	9.15	7.49
TÜLU 2+DPO 70B	9.00	**9.90**	**7.00**	4.70	4.65	**9.35**	**9.25**	**9.25**	**7.89**

Table 8: Full MT-Bench results split by category. Score is an average of scores given by a GPT-4 annotator. The best open-weight model performance is underlined.